

Rumo

Banco de dados

O que o navegador guarda hoje, campo a campo, e a arquitetura sugerida em PostgreSQL para quando o catálogo sair dos arquivos TypeScript, com contas e progresso sincronizado: diagrama, tabelas, índices, DDL, fluxos de dados, operação e como escalar.

Copyright (©) 2026 Erik Pimenta
Versão online: https://ravelroot.github.io/Circuit-Board-Bench-Lab/

Versão 1.3.0 · Novembro de 2026 · Todos os direitos reservados

Versão 1.3.0 · Novembro de 2026

Novidades da versão 1.3.0

A versão 1.3.0 leva o catálogo de 32 para **132 placas** e cria **duas categorias**: Sensores e Energia. No modelo sugerido, isso é dado, não estrutura: a tabela `categoria` ganha duas linhas, a restrição da coluna `chave` aceita os dois valores novos e a ordem do catálogo é renumerada. Nenhuma tabela de progresso muda, e o que cada pessoa marcou continua valendo, porque o progresso aponta para slugs que não mudaram.

> **Onde está:** os valores novos aparecem marcados com NOVO em "Tabelas" (categoria) e em "DDL principal", a migração em "DDL principal" e "Operação e escala", e o volume novo do catálogo em "Quando migrar".

Onde	O que muda
`categoria`	Linhas `sensor` (Sensores) e `energia` (Energia); `ordem` passa a ser 1 a 7: microcontrolador, computador, pc, modulo, sensor, energia e circuito
`placa`	Na versão 1.3.0, `hcsr04`, `mpu6050` e `neo6m` apontam para `sensor`, e `lm2596` e `tp4056` para `energia`. As versões anteriores, imutáveis, continuam como eram
Volume	132 placas, 921 partes, 1.850 componentes e 1.783 footprints de PCB numa versão publicada
Progresso	Nenhuma mudança: slugs de placas, peças e labs são os mesmos
Migrações	Novo arquivo `005_categorias_sensor_energia.sql`

Versão 1.2.1 · Novembro de 2026

Novidades da versão 1.2.1

A versão 1.2.1 só mudou a interface: o menu de paletas passou a mostrar apenas as amostras de cor. Nenhuma tabela, coluna, enum, índice ou restrição mudou.

Versão 1.2.0 · Novembro de 2026

Novidades da versão 1.2.0

A versão 1.2.0 acrescentou a paleta de cores escolhida por quem usa: o campo `paleta` em `rumo:preferencias` e, no modelo sugerido, a coluna `paleta` em `usuario`, com o enum `paleta` e a migração `004_usuario_paleta.sql`.

Sumário

Quando migrar 4

O que o navegador guarda hoje 4

Decisões 5

Diagrama de relacionamentos 6

Entidades 6

Tabelas 7

versao_catalogo 7

categoria 7

placa 8

especificacao, parte e componente 8

tema e peca 9

laboratorio 9

Listas e relações de um lab 9

projeto_pcb 10

usuario 10

bancada_item, passo_marcado e lab_concluido 10

Índices 11

DDL principal 12

Migração 005: Sensores e Energia 13

Fluxos de dados 14

Leitura do catálogo 14

Gravação do progresso e das preferências 14

Operação e escala 15

Operação 15

Como escalar 15

Licença 15

Quando migrar

Hoje o Rumo não precisa de banco: o catálogo (132 placas em sete categorias, 921 partes, 1.850 componentes, 1.783 footprints de PCB e 25 labs) é compilado junto do site e as preferências e o progresso ficam no navegador. No build, o catálogo já sai num arquivo próprio, de cerca de 125 KB comprimido. A migração vale a pena quando surgir pelo menos uma destas necessidades:

- editar placas e labs por um painel, sem novo deploy;
- contas para levar bancada, passos, labs feitos e preferências para outros aparelhos;
- turmas, em que um professor acompanha o progresso dos alunos;
- métricas de uso para decidir quais placas e labs criar.

Mesmo com banco, o site continua estático: um job exporta a versão publicada para um JSON por versão, servido por CDN. O banco só é consultado pela API de progresso e pelo painel de edição.

O que o navegador guarda hoje

Chave	Campo	Valores aceitos
`rumo:preferencias`	`idioma`	`pt` ou `en`; sem valor salvo, segue o navegador
`rumo:preferencias`	`tema`	`claro` ou `escuro`; sem valor salvo, segue o sistema
`rumo:preferencias`	`paleta`	`rumo`, `bento`, `sonda` ou `nexo`; sem valor salvo, usa a paleta Rumo
`rumo:progresso`	`bancada`	Ids de peças que existem no catálogo, sem repetição
`rumo:progresso`	`passos`	Por lab, os números dos passos marcados, dentro do intervalo
`rumo:progresso`	`feitos`	Ids de labs concluídos

A leitura descarta qualquer valor desconhecido, repetido ou longo demais, então um dado corrompido nunca quebra a página. A rota da página (por exemplo `#/esp32-s3`) não é guardada: endereço vazio abre a placa inicial, a primeira do catálogo.

Decisões

Decisão	Motivo
PostgreSQL 16	Integridade referencial, enums, JSONB com validação, busca textual em português e inglês e operação conhecida
Textos em colunas `_pt` e `_en`	São só duas línguas e todas são obrigatórias; colunas NOT NULL garantem que nenhum texto fica faltando, o que uma tabela genérica de traduções não garante
Categorias em tabela, com CHECK	A ordem e os nomes nas duas línguas ficam no banco; uma categoria nova é um INSERT e uma troca de CHECK, sem reescrever `placa`, como aconteceu com Sensores e Energia
Componentes em tabela, não em JSONB	A ligação componente → parte precisa de chave estrangeira, e os testes de "toda parte desenhada" viram uma consulta
Configuração da prévia em JSONB	Cada um dos 16 simuladores tem um formato diferente; o JSONB é validado por JSON Schema por `previa_tipo` antes de gravar
Versões publicadas e imutáveis	Edições vão para um rascunho; publicar congela a versão. Voltar é só apontar para a anterior
Projeto de PCB como dado derivado	Footprints, pads, trilhas e vias saem do gerador a partir dos componentes. Guardar o resultado em `projeto_pcb` (JSONB com hash dos componentes) evita duplicar a fonte da verdade
Progresso por slug	O progresso aponta para `laboratorio_slug` e `peca_slug`, que não mudam entre versões, então uma nova publicação não apaga o que o usuário marcou
Preferências em colunas com enum	Idioma, tema e paleta são poucos e fechados; um enum por campo garante que a conta nunca guarda um valor que o site não sabe aplicar

Diagrama de relacionamentos

Entidades

Em linho, o catálogo versionado; em rosé, os dados do usuário. Uma versão tem placas e peças; cada placa pertence a uma categoria e tem especificações, partes, componentes, temas, labs e o projeto de PCB gerado. Os dados do usuário apontam para slugs estáveis, não para ids de versão.

versao_catalogo

PK	id	bigint
	numero	text
	estado	enum
	publicada_em	timestamptz

categoria

PK	id	smallint
	chave	text
	nome_pt / nome_en	text
	ordem	smallint

placa

PK	id	bigint
FK	versao_id	bigint
FK	categoria_id	smallint
	slug, nome, sigla	text
	largura_mm, altura_mm	numeric
	cor_pcb, ordem	enum, smallint

especificacao

PK	id	bigint
FK	placa_id	bigint
	ordem	smallint
	rotulo, valor pt/en	text

parte

PK	id	bigint
FK	placa_id	bigint
	slug	text
	nome, oque, funcao	text
	ordem	smallint

componente

PK	id	bigint
FK	placa_id	bigint
FK	parte_id	bigint
	forma	enum
	x, y, l, a, z (mm)	numeric

tema

PK	id	bigint
FK	placa_id	bigint
	slug	text
	nome, resumo	text

peca

PK	id	bigint
FK	versao_id	bigint
	slug	text
	nome_pt / nome_en	text

laboratorio

PK	id	bigint
FK	placa_id	bigint
FK	tema_id	bigint
	slug, numero	text, int
	previa_tipo	enum
	previa_config	jsonb

laboratorio_peca / _parte

PK FK	laboratorio_id	bigint
PK FK	peca_id / parte_id	bigint

bloco_como / ligacao / comando / passo

PK	id	bigint
FK	laboratorio_id	bigint
	ordem	smallint
	textos pt/en	text

projeto_pcb

PK FK	placa_id	bigint
	gerador_versao	text
	conteudo	jsonb
	hash, gerado_em	char, tstz

usuario

PK	id	uuid
	email	citext
	idioma, tema	enum
	paleta	enum
	excluido_em	timestamptz

bancada_item

PK FK	usuario_id	uuid
PK	peca_slug	text
	marcado_em	timestamptz

passo_marcado / lab_concluido

PK FK	usuario_id	uuid
PK	laboratorio_slug	text
PK	passo_ordem	smallint
	concluido_em	timestamptz

Tabelas

versao_catalogo

Cada publicação do catálogo é uma versão imutável. O site lê sempre a última publicada.

Campo	Tipo	Chaves e restrições
`id`	bigint identity	PK
`numero`	text	UNIQUE, semver (ex.: 1.3.0)
`estado`	`estado_versao`	`rascunho`, `publicada` ou `arquivada`
`publicada_em`	timestamptz	NULL até publicar
`notas`	text	Notas da versão

categoria

Campo	Tipo	Chaves e restrições
`id`	smallint	PK
`chave`	text	UNIQUE, CHECK in (`microcontrolador`, `computador`, `pc`, `modulo`, `sensor`, `energia`, `circuito`) NOVO
`nome_pt`, `nome_en`	text	NOT NULL
`ordem`	smallint	NOT NULL, UNIQUE; a ordem dos grupos no catálogo

Linhas da tabela na versão 1.3.0:

id	chave	nome_pt	nome_en	ordem
1	`microcontrolador`	Microcontroladores	Microcontrollers	1
2	`computador`	Computadores de placa única	Single-board computers	2
3	`pc`	Hardware de PC	PC hardware	3
4	`modulo`	Módulos	Modules	4
6	`sensor` NOVO	Sensores	Sensors	5
7	`energia` NOVO	Energia	Power	6
5	`circuito`	Circuitos	Circuits	7

placa

Campo	Tipo	Chaves e restrições
`id`	bigint identity	PK
`versao_id`	bigint	FK `versao_catalogo` ON DELETE CASCADE
`categoria_id`	smallint	FK `categoria`
`slug`	text	UNIQUE (`versao_id` , `slug`), CHECK `slug ~ '^[a-z0-9-]{1,48}$'`
`nome` , `sigla`	text	NOT NULL
`fabricante_pt` , `fabricante_en`	text	
`resumo_pt/en` , `usos_pt/en` , `essenciais_pt/en`	text	NOT NULL
`largura_mm` , `altura_mm`	numeric(7,2)	CHECK > 0
`espessura_mm`	numeric(4,2)	DEFAULT 1.6
`cor_pcb`	`cor_pcb`	enum: verde, azul, preto, vermelho, roxo, branco, amarelo, cinza, marrom
`ordem`	smallint	Posição no catálogo; a de menor ordem é a página inicial (hoje, a ESP32-S3)

especificacao, parte e componente

Campo	Tipo	Chaves e restrições
`especificacao.placa_id` , `ordem`	bigint, smallint	FK `placa` ON DELETE CASCADE; UNIQUE (`placa_id` , `ordem`)
`especificacao.rotulo_pt/en` , `valor_pt/en`	text	NOT NULL
`parte.placa_id` , `slug`	bigint, text	FK `placa` ; UNIQUE (`placa_id` , `slug`)
`parte.nome_pt/en` , `oque_pt/en` , `funcao_pt/en`	text	NOT NULL
`componente.parte_id`	bigint	FK `parte` , NULL para decoração
`componente.forma`	`forma_peca`	enum com as 30 formas
`componente.x_mm` , `y_mm` , `largura_mm` , `altura_mm` , `z_mm`	numeric	Largura e altura > 0, z >= 0
`componente.rotulo` , `cor`	text	NULL; cor CHECK `~ '^#[0-9a-f]{6}$'`

tema e peca

Campo	Tipo	Chaves e restrições
`tema.placa_id`, `slug`	bigint, text	FK `placa`; UNIQUE (`placa_id`, `slug`)
`tema.nome_pt/en`, `resumo_pt/en`	text	NOT NULL
`peca.versao_id`, `slug`	bigint, text	FK `versao_catalogo`; UNIQUE (`versao_id`, `slug`)
`peca.nome_pt`, `nome_en`	text	NOT NULL

laboratorio

Campo	Tipo	Chaves e restrições
`id`	bigint identity	PK
`placa_id`, `tema_id`	bigint	FK `placa` ON DELETE CASCADE, FK `tema`
`slug`	text	UNIQUE (`placa_id`, `slug`)
`numero`	smallint	UNIQUE (`placa_id`, `numero`)
`titulo_pt/en`, `resumo_pt/en`, `explicacao_previa_pt/en`	text	NOT NULL
`fiacao_modulo`	text	NULL quando não há fios
`previa_tipo`	`tipo_previa`	enum com os 16 simuladores
`previa_config`	jsonb	Validado por JSON Schema por tipo
`codigo_arquivo`, `codigo_texto`	text	NULL quando não há código

Listas e relações de um lab

Tabela	Campos	Chaves e restrições
`laboratorio_peca`, `laboratorio_parte`	`laboratorio_id`, `peca_id` / `parte_id`	PK composta, FKs ON DELETE CASCADE
`bloco_como`, `passo`	`ordem`, `texto_pt/en`	UNIQUE (`laboratorio_id`, `ordem`)
`ligacao`	`pino_modulo`, `pino_placa`, `cor_fio`, `nota_pt/en`	`cor_fio` enum com 10 cores
`comando`	`rotulo_pt/en`, `linha`	NOT NULL

projeto_pcb

Cache do projeto de PCB gerado para uma placa. O projeto é derivado e determinístico: o gerador recria o mesmo resultado a partir dos componentes, então guardar é opcional e serve para servir o JSON pronto, comparar versões e auditar. Footprints, pads, trilhas e vias só viram tabelas normalizadas se o PCB passar a ser editado à mão. Com 132 placas, são 1.783 footprints e mais de 21 mil pads por versão, o que cabe com folga em 132 linhas de JSONB.

Campo	Tipo	Chaves e restrições
`placa_id`	bigint	PK, FK `placa` ON DELETE CASCADE
`gerador_versao`	text	NOT NULL, versão do gerador
`conteudo`	jsonb	NOT NULL, formato `ProjetoPcb`: footprints, trilhas, vias, furos, textos e antenas
`hash`	char(64)	NOT NULL, SHA-256 dos componentes que geraram o projeto
`gerado_em`	timestamptz	DEFAULT `now()`

usuario

Conta opcional para sincronizar o progresso e as preferências.

Campo	Tipo	Chaves e restrições
`id`	uuid	PK DEFAULT `gen_random_uuid()`
`email`	citext	UNIQUE
`idioma`	`idioma`	enum: `pt`, `en`
`tema`	`tema`	enum: `claro`, `escuro`; NULL segue o sistema
`paleta`	`paleta`	enum: `rumo`, `bento`, `sonda`, `nexo`; DEFAULT `rumo`
`criado_em`	timestamptz	DEFAULT `now()`
`excluido_em`	timestamptz	Exclusão lógica, LGPD

bancada_item, passo_marcado e lab_concluido

Progresso do usuário, ligado a slugs estáveis para sobreviver a novas versões do catálogo.

Campo	Tipo	Chaves e restrições
`usuario_id`	uuid	FK `usuario` ON DELETE CASCADE
`peca_slug` / `laboratorio_slug`	text	Parte da PK
`passo_ordem`	smallint	Parte da PK em `passo_marcado`
`marcado_em` / `concluido_em`	timestamptz	DEFAULT `now()`

Índices

Índice	Uso
`placa (versao_id, ordem)`	Lista do catálogo na ordem certa e a placa inicial
`placa (versao_id, categoria_id)`	Grupos do catálogo e chips de troca de placa
`parte (placa_id, ordem)`	Cartões da vista 3D
`componente (placa_id, ordem)`	Desenho da placa num único índice
`laboratorio (placa_id, numero)`	Mapa de labs e próximo/anterior
`laboratorio_peca (peca_id)`	Em quantos labs uma peça aparece
GIN em `to_tsvector(nome, resumo, partes)`	Busca textual nas duas línguas (configurações `portuguese` e `english`)
`bancada_item (usuario_id)`	Bancada do usuário; a PK já cobre
`passo_marcado (usuario_id, laboratorio_slug)`	Passos de um lab
`projeto_pcb (gerador_versao)`	Achar projetos gerados por uma versão antiga do gerador para regenerar

DDL principal

```
CREATE TYPE estado_versao AS ENUM ('rascunho', 'publicada', 'arquivada');
CREATE TYPE cor_pcb AS ENUM
  ('verde','azul','preto','vermelho','roxo','branco','amarelo','cinza','marrom');
CREATE TYPE idioma AS ENUM ('pt', 'en');
CREATE TYPE tema AS ENUM ('claro', 'escuro');
CREATE TYPE paleta AS ENUM ('rumo', 'bento', 'sonda', 'nexo');
```

```
CREATE TABLE versao_catalogo (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  numero text NOT NULL UNIQUE,
  estado estado_versao NOT NULL DEFAULT 'rascunho',
  publicada_em timestamptz
);

CREATE TABLE categoria (
  id smallint PRIMARY KEY,
  chave text NOT NULL UNIQUE CONSTRAINT categoria_chave_valida CHECK (chave IN
    ('microcontrolador','computador','pc','modulo','sensor','energia','circuito')),
  nome_pt text NOT NULL,
  nome_en text NOT NULL,
  ordem smallint NOT NULL UNIQUE DEFERRABLE INITIALLY IMMEDIATE
);
```

```
CREATE TABLE placa (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  versao_id bigint NOT NULL REFERENCES versao_catalogo ON DELETE CASCADE,
  categoria_id smallint NOT NULL REFERENCES categoria,
  slug text NOT NULL CHECK (slug ~ '^[a-z0-9-]{1,48}$'),
  nome text NOT NULL,
  sigla text NOT NULL,
  resumo_pt text NOT NULL, resumo_en text NOT NULL,
  largura_mm numeric(7,2) NOT NULL CHECK (largura_mm > 0),
  altura_mm numeric(7,2) NOT NULL CHECK (altura_mm > 0),
  espessura_mm numeric(4,2) NOT NULL DEFAULT 1.6,
  cor_pcb cor_pcb NOT NULL,
  ordem smallint NOT NULL,
  UNIQUE (versao_id, slug)
);

CREATE TABLE projeto_pcb (
  placa_id bigint PRIMARY KEY REFERENCES placa ON DELETE CASCADE,
  gerador_versao text NOT NULL,
  conteudo jsonb NOT NULL CHECK (jsonb_typeof(conteudo -> 'footprints') = 'array'),
  hash char(64) NOT NULL,
  gerado_em timestamptz NOT NULL DEFAULT now()
);
```

```
CREATE TABLE usuario (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  email citext NOT NULL UNIQUE,
  idioma idioma NOT NULL DEFAULT 'pt',
  tema tema,
  paleta paleta NOT NULL DEFAULT 'rumo',
  criado_em timestamptz NOT NULL DEFAULT now(),
  excluido_em timestamptz
);

CREATE TABLE lab_concluido (
  usuario_id uuid NOT NULL REFERENCES usuario ON DELETE CASCADE,
  laboratorio_slug text NOT NULL,
  concluido_em timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (usuario_id, laboratorio_slug)
);
```

Migração 005: Sensores e Energia

A migração roda numa transação. Ela troca a restrição, cria as duas linhas e renumera a ordem; as placas da versão em rascunho passam para as categorias novas pelo slug. Versões já publicadas não são tocadas.

```
BEGIN;
ALTER TABLE categoria DROP CONSTRAINT categoria_chave_valida;
ALTER TABLE categoria ADD CONSTRAINT categoria_chave_valida CHECK (chave IN
  ('microcontrolador','computador','pc','modulo','sensor','energia','circuito'));
SET CONSTRAINTS ALL DEFERRED;
INSERT INTO categoria (id, chave, nome_pt, nome_en, ordem) VALUES
  (6, 'sensor', 'Sensores', 'Sensors', 5),
  (7, 'energia', 'Energia', 'Power', 6);
UPDATE categoria SET ordem = 7 WHERE chave = 'circuito';
UPDATE placa p SET categoria_id = c.id
  FROM categoria c, versao_catalogo v
  WHERE p.versao_id = v.id AND v.estado = 'rascunho'
    AND ((c.chave = 'sensor' AND p.slug IN ('hcsr04', 'mpu6050', 'neo6m'))
      OR (c.chave = 'energia' AND p.slug IN ('lm2596', 'tp4056')));
COMMIT;
```

Fluxos de dados

Leitura do catálogo

1. O painel publica uma versão: `estado = 'publicada'` e `publicada_em = now()`, numa transação.
2. Um job exporta a versão para `catalogo-1.3.0.json`, no mesmo formato dos tipos TypeScript, e envia para a CDN com cache longo.
3. O site busca `catalogo.json` (cache curto, aponta para a versão) e depois o arquivo versionado. A primeira placa pela `ordem` é a página inicial, para onde o logo leva.
4. No mesmo job, o gerador de PCB roda para cada placa cujo hash de componentes mudou e grava `projeto_pcb`.

Gravação do progresso e das preferências

1. Sem conta, tudo continua no `localStorage`, como hoje: `rumo:preferencias` e `rumo:progresso`.
2. Ao entrar, o navegador envia o progresso e as preferências locais; o servidor faz a união do progresso (`INSERT ... ON CONFLICT DO NOTHING`), grava idioma, tema e paleta e devolve o estado combinado.
3. Em outro aparelho, a resposta do login traz as preferências, que o site grava no `localStorage` antes de aplicar o tema e a paleta.
4. Cada marcação depois disso vira um `PUT` idempotente, com fila local para quando estiver sem rede.

Operação e escala

Operação

- Migrações versionadas no repositório e aplicadas no CI antes do deploy da API; a 1.3.0 entra com `005_categorias_sensor_energia.sql`, depois de `004_usuario_paleta.sql`.
- Backup diário com retenção de 30 dias e recuperação ponto a ponto pelo WAL.
- LGPD: o único dado pessoal é o e-mail; a exclusão da conta apaga progresso em cascata e anonimiza o usuário.
- Acesso: a API usa um papel só com permissão nas tabelas de progresso e preferências; o painel usa outro papel com escrita só em versões em rascunho.
- Segredos em variáveis de ambiente do provedor, nunca no repositório.

Como escalar

- O catálogo não pesa no banco: é lido da CDN. O banco só atende escrita de progresso, leve e por chave primária.
- O JSON publicado cresce com o catálogo. Com algumas centenas de placas, vale dividir por categoria e carregar cada grupo quando ele aparece, como o site já faz ao separar o three.js e o catálogo em arquivos próprios.
- Com muitos usuários, `passo_marcado` pode ser particionada por hash de `usuario_id`.
- Réplicas de leitura atendem os painéis de professores e as métricas.
- Uma categoria nova é um INSERT e uma troca de CHECK; uma paleta nova só acrescenta um valor ao enum `paleta` (`ALTER TYPE paleta ADD VALUE`), sem reescrever a tabela.
- Métricas de uso vão para uma tabela de eventos separada, particionada por mês, ou para um armazém de dados.

Licença

Copyright (©) 2026 Erik Pimenta. Todos os direitos reservados. Este documento e o Rumo seguem a licença de uso do arquivo `LICENSE`: é permitido apenas usar a versão publicada e baixar o projeto para rodar no próprio computador, sem alterações. Modificar, criar obras derivadas, vender, cobrar pelo acesso, redistribuir, hospedar em outro endereço ou reutilizar o modelo de dados, o código, os textos, o nome ou a logo depende de autorização por escrito do autor.